

PART 2

Dopo aver studiato la computabilità di un problema, ovvero se è decidibile o meno, passiamo a studiare la sua complessità: studiamo se può essere risolto in maniera efficiente o meno. Si fa distinzione tra prob. trattabili e non.

La prima distinzione si fa tra i linguaggi decidibili, sono tutti uguali in efficienza? No

- I regulari vengono eseguiti in una sola scansione (lineare).
- I CFL scan tempo polinomiale ed usano un $\log^2$ di celle blank
- Linguaggi hard = esiste TM che scansiona l'input in modo esp?

Tra i linguaggi hard la prima distinzione è tra

- POLINOMIALI $\rightarrow$ tutti quelli che vengono risolti in $O(n^k)$
- NON POLINOMIALI (esp) maggiore $O(n^k)$
- I primi sono detti TRATTABILI, secondi no

La prima cosa da analizzare è il tempo.

Esso viene valutato in base alla funzione TIMEREQ.

Misura la complessità temporale di una TM in base a:

- Se STM è il numero di step massimo che esegue su input N
- Se NSTM è il numero massimo di step nel path più lungo su input N

Lo spazio invece viene misurato tramite la funzione SPACEREQ, che misura il numero di celle massime toccate

- STM = numero di celle massime ~~non~~ ^{si legge su n} ~~per ogni passo~~
- NSTM = numero di celle massime ~~nel path più lungo~~ ^{si legge su n}

Queste funzioni vengono associate ad alcune funzioni asintotiche

$O(g) \rightarrow f$ al massimo d'ordine g | $\phi(g) = \text{if } \exists k \in \mathbb{N} \forall n \geq k (f(n) < c g(n))$
 $\Omega(g) \rightarrow f$ al minimo ordine g
 $\Theta(g(n)) \rightarrow f$ è di ordine g

L'obiettivo è quello di caratterizzare i problemi in base alla complessità.
Per alcuni problemi non esiste alcuna ottimizzazione, esiste cioè
un gap tra lower and upper. Questi problemi si definiscono aperti.

In base a queste caratteristiche abbiamo alcune classi.

P-TIME.

È l'insieme dei linguaggi decisi da un TM in tempo polinomiale.

P-SPACE $\rightarrow \exists TM$ $L(n)$ e lo spazio richiesto è $O(n^k)$.

EXPTIME $\rightarrow \exists TM L(n)$ tempo $O(2^{n^k})$

EXPSPACE $\rightarrow \exists TM L(n)$ spazio $O(2^{n^k})$

⋮

ecc

Relazione

La relazione è che se sappiamo che un L è deciso in
tempo polinomiale allora avremo bisogno di al massimo
spazio polinomiale

$P(TIME) \subseteq P(SPACE)$ NB in n tempo, controllo
al massimo N celle.

Invece la relazione tra lo spazio e il tempo è diversa,
infatti se abbiamo n celle, avremo bisogno di almeno n
tempo, ed al massimo 2^{n^k} tempo se ci sono modifiche 2^{n^k}

Dunque:

$PSPACE \subseteq EXPTIME$

D-TIME

E' l'insieme dei linguaggi per cui:

- $\exists TM M$ s.t che decide L

- $Time_{dec}(M) \in O(n^k)$

Dunque se $L \in P$ è fattibile

PROPRIETA'

(chiusa sotto $\bar{\cdot}, \cup, \cap, Kleen^*$ ^{grafo.}

NO Ogni linguaggio REG, CFL $\in P$ + Bonus.

Per verificare che un linguaggio è in P abbiamo bisogno di un algoritmo che lo risolva in tempo polinomiale.

ASI VISTI: BFS, DFS, $|A| = |V| - 1$
Reachability, Connected, Tree, MST, MST, ^{grafo connesso} $\rightarrow$ KROUSCAL

KROUSCAL

- ordina edges in base al peso
- Inizializzo lista T
- per ogni arco $\rightarrow$ verifico che a, b non siano ancora int.
; se lo aggiungo
- Restituisco T

NBDM

È una sestupla $(K, \Sigma, \Gamma, S, H, \Delta)$

La diff con DTM è Δ , mentre in S ogni conf ha solo un succ
nelle Δ vi sono più successori per ogni conf.

Dunque non è una funzione ma una relazione:

$$((K-H) \times \Gamma) \times (K \cdot \Gamma \times \{\leftarrow, \rightarrow\})$$

NBDM ~~accetta~~ ^{parla} un linguaggio ~~se e solo se~~

M accetta se e solo se raggiunge yes in un path

M rifiuta se e solo se M raggiunge no in un path

M decide se e solo se

- $W \in L$ e M accetta

- Esiste un numero finito di path da cui si arresta su yes

- Ogni path si arresta

Si naviga tramite BFS

Per ogni NBDM è possibile costruire una DTM.

$NBDM = DTM$

- NBDM è un grafo finito $\Rightarrow G(N, A)$

- DTM scrive Δ sul nastro.

- DTM usa DFS per verificare se si arresta e accetta.

Poiché

EXP-TIME
 $\uparrow$

DTM imita NBDM $\text{space}(NBDM)$ allora $DTM \in \textcircled{2^{hk}}$

$P \subset N \subset EX$

~~P-TIME~~ ~~SPACE~~ $\sim$ $P-TIME \subset PSPACE \subset EXPTIME$

NP.

La classe di linguaggi NP è composta da quei linguaggi che possono essere risolti in tempo polinomiale.

$L \in NP$ iff $\exists NDTM M$ $L(M)$ in $time(\pi)$ è polinomiale.

Abbiamo diversi modi per verificare che $L \in NP$

- 1- costruire una NDTM che decide ^{polinomiale in n}
- 2- Se abbiamo una soluzione candidata c con $|c| \leq O(n^k)$ (è una TM det)
- 3- Guess & Check (si crea una soluzione on the fly).
- 4- Riduzione con linguaggi in NP

Linguaggi in NP

- SAT
- 3SAT
- $\neg$ SAT
- HP
- Knapsack
- set partition
- clique
- IS

Non sappiamo se $P = NP$ però siamo certi che

$$P \subseteq NP$$

Reduction

Una mapping reduction è una funzione f che mappa le istanze di A in istanze di B .

$A \leq_p B$ indica che se $A \in \text{NP}$ allora $B \in \text{NP}$

È una riduzione polinomiale se f è polinomiale

MEMBERSHIP: $L_1 \subseteq_p L_2$ se $L_2 \in \text{P}$ allora $L_1 \in \text{P}$

WARMUP: $L_1 \not\subseteq_p L_2$ dimostrano che L_1 non è più ^{complesso di} L_2

$3\text{-SAT} \subseteq_p \text{IND-SET}$ → utilizza i gadget

↓
struttura da simulare L_1 ...
in forma di L_2

Polynomial Hierarchy. (Generalizza P, NP , e $coNP$ tramite oracolo)

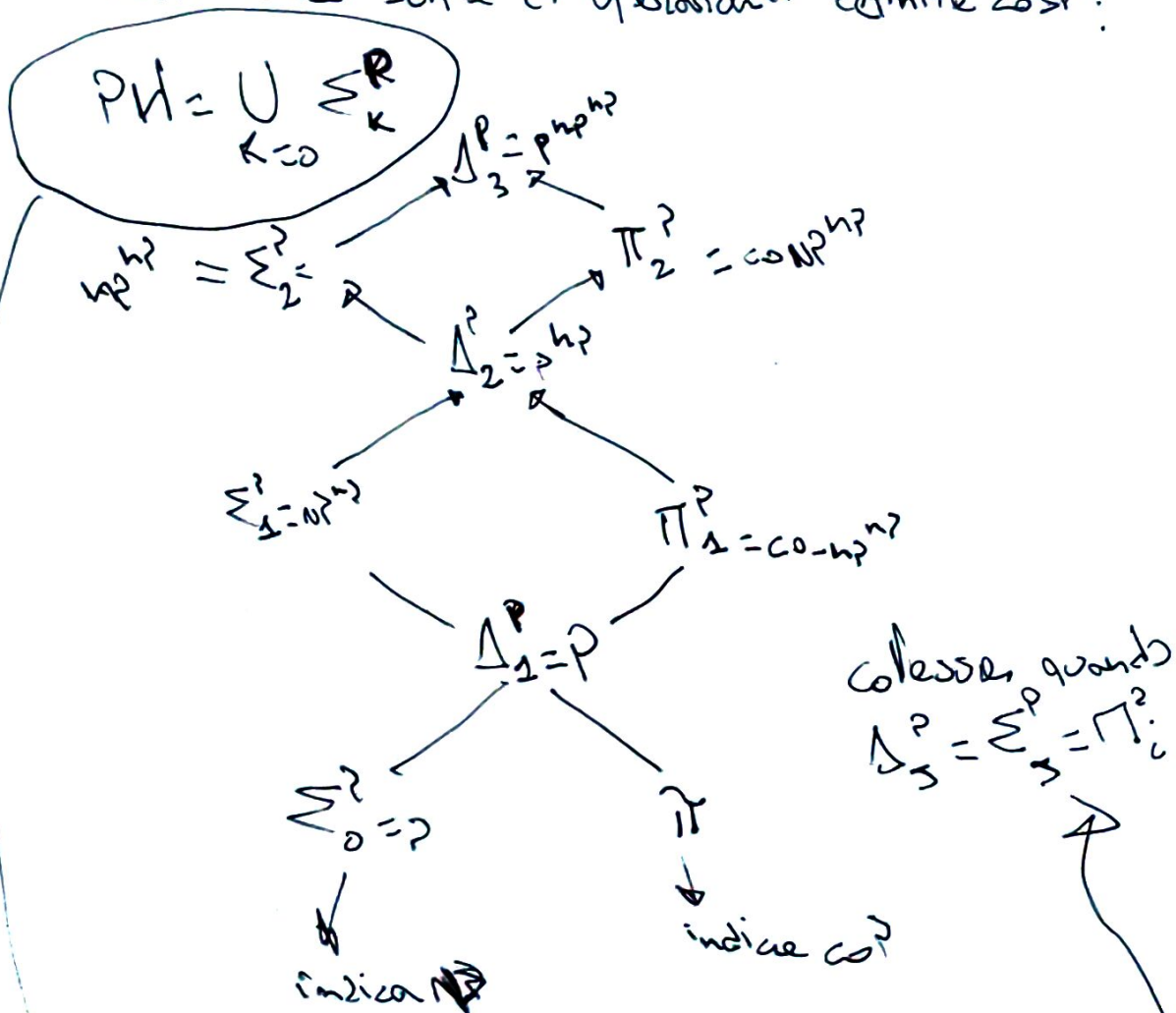
Vi sono 2^{te} problemi che non possono essere risolti in NP .
Un esempio È **UNIQUE SAT**.

Non sappiamo se quelli questi sono giusti.

Si usa l'oracolo, un subsystem che risolve il problema in NP , 1.

Avremo una procedura che lavora in NP^{NP} che a sua volta
deve usare così chiamare un altro oracolo

Avremo una sorta di gerarchia definite così:



→ Sono immediatamente sotto $PSPACE$

~~$PH \subseteq PSPACE$~~

Hence $P \subseteq NP \subseteq PH \subseteq PSPACE$

LOG-SPACE

$$n^k \cdot \log n \rightarrow$$

All'interno di P abbiamo REG che lavora in spazio costante
cerchiamo tra REG e P

troviamo la classe LOG, ovvero quelle classi che usano spazio costante, ovvero non utilizzano più spazio di quello fornito in input. Utilizziamo più tape in cui nelle tape si può mantenere tutto quello che serve per lavorare tra le stringhe in input senza accedere alla memoria (contatori, espressioni ecc.).

Le operazioni che possono usare sono:

- Inizializzazione
- Assignment
- Increment - decrement
- Comparisio
- Operazioni Matematiche.

$$REG \subseteq LOG \subseteq P$$

IN LOG abbiamo:

- Deterministic Reach
- Circuit

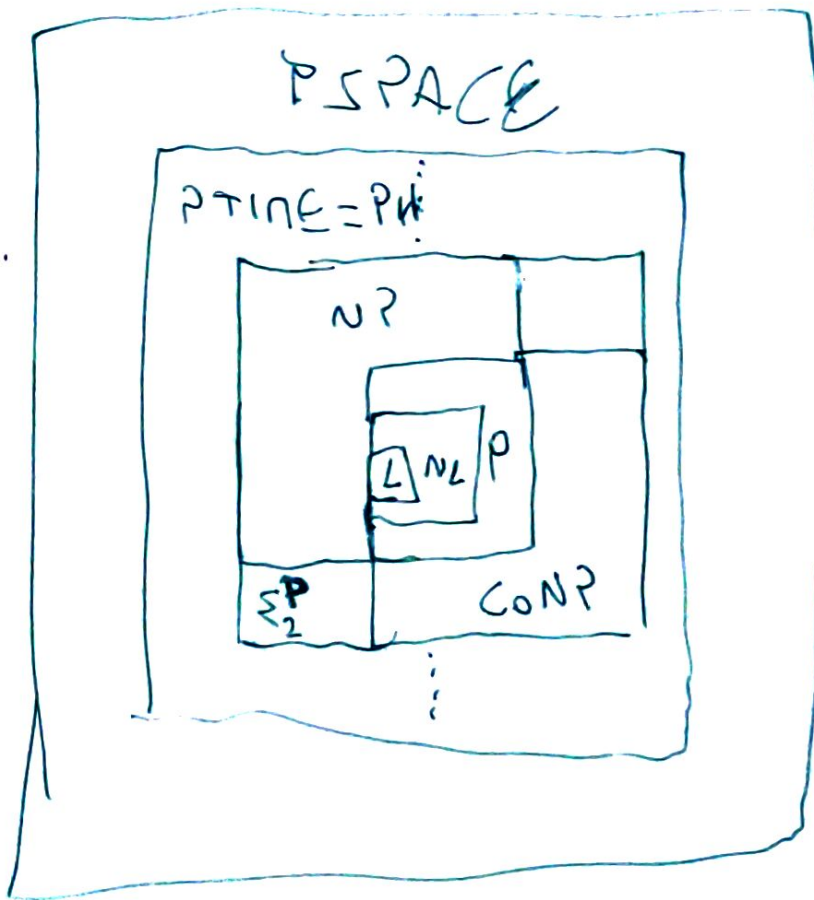
TRA LOG e P?

NL

Sono i problemi che sono decidibili da una NDTM in
~~tempo~~ spazio logaritmico

Resumé final.

~~REG~~ $L \subseteq NL \subseteq P \subseteq NP \subseteq PH \subseteq PSPACE \subseteq EXP-TIME \subseteq EXPSPACE$



$\vdots$
 D/C

1. Reachability

Deve esistere un path da u a v e t

2 method:

BFS $\rightarrow$ a livelli

DFS $\rightarrow$ in profondità

reach $\in P$

2. Connected $\in P$

G is UG ed è connesso (esiste un path per ogni $\{u, v\} \in E$)

fasi:

1- Da UG a UG

2- Applica BFS

Tree $\in P$

fasi

1- G è connesso

2- $|A(G)| = |V| - 1$

(G^2)

il più piccolo sottografo
connected

3. Minimum Spanning Tree $\in P \rightarrow V' \subseteq V, E' \subseteq E$ è connesso $G \in DG$ weighted

- Si crea con KRUSKAL

1- ordine gli archi per costo

2- Inizializza T lista vuota

3- Finché non sono tutti gli E selezionati $e \in E$

- confronta gli estremi, se non connessi in T aggiungi

4- ridona T

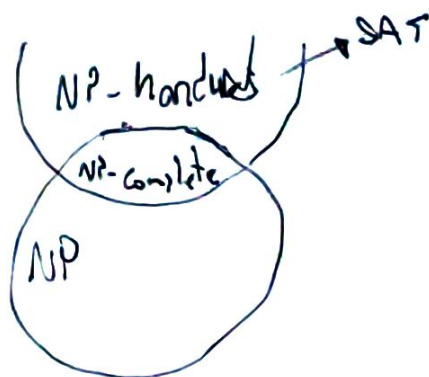
NP-hard

Un linguaggio L è detto NP-hard se è difficile almeno quanto ogni altro in NP. Dunque deve esistere una riduzione polinomiale da $L_i \leq L$.

NP-complete

Un linguaggio L è in NP-complete se ogni linguaggio L' che appartiene ad NP può essere ridotto a L e se $L \in NP$.

Dunque è l'intersezione tra NP-hard e NP



Per dimostrare che un linguaggio L è NP-complete bisogna dimostrare:

1- $L \in NP$ (tramite: Verifier + certificate)

- guess-check
- reduction con Linguaggio in NP
- macchine non deterministiche

2- L è NP-hard $\rightarrow$ Bisogna provare che esiste una riduzione polinomiale tra L e ogni $L_i \in NP$.

Usiamo SAT, poiché è stato provato che SAT è NP-HARD.

Cook LEVIN

NP-complete

Ha dimostrato che SAT è NP-hard, ovvero che ogni linguaggio è NP è riducibile ad esso: $L_i \leq_p \text{SAT}$

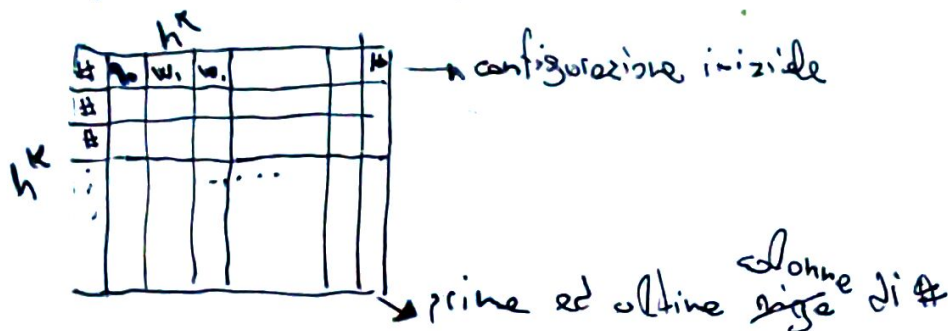


Se $w \in L \rightarrow \langle y \rangle \in \text{SAT}$

Se $w \notin L \rightarrow \langle y \rangle \notin \text{SAT}$

La riduzione funzione fornisce una matrice delle TABLEAU, in cui ogni riga corrisponde ad una configurazione della M.

Le dimensioni sono $n^k \times n^k$ (numero massimo di step per formarsi).



Il Tableau è accettabile se si giunge ad una riga con configurazione di accettazione. Per dimostrare SAT si trasforma questo TABLEAU in una formula Φ .

1. Ogni cella avrà una variabile binaria $x_{i,j,s}$ (dove i è la riga, j la colonna, s il simbolo).
 2. In ogni cella deve esserci solo un simbolo (almeno e uno).

3. Le prime righe dove esiste la configurazione di partenza.

4. Almeno una riga deve essere la configurazione d'accettazione.

5. Garantisce che lo spostamento avviene in modo legale, ogni configurazione deve essere seguita da una corretta. Si usa 2 o 3 variabili.

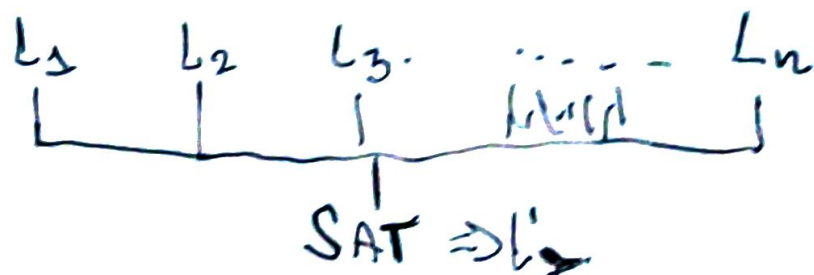
$\Phi = \Phi_{\text{cell}} \wedge \Phi_{\text{start}} \wedge \Phi_{\text{move}} \wedge \Phi_{\text{cell}}$



La formula finale dunque sarà 1 solo se tutte queste saranno 1
per costruzione abbiamo fatto in modo che ciò avvenga l'unica volta

↑ All questo avviene in tempo Polinomiale $2^{2k} n^{2k}$,
dunque SAT è ~~NP-hard~~, NP-Complete.

Dunque:



NP-COMPLETE CONOSCIUTI:

- SAT e CIRCUIT-SAT
- CNF-SAT
- 3SAT
- TSP
- KNAPSACK
- CLIQUE
- VERTEX COVER
- INDEPENDENT SET
- 3-COLORING

NP-HARD CONOSCIUTI

- SAT
- HALTING PROBLEM
-

Abbiamo trovato un problema L' tale per cui ogni
linguaggio può essere ridotto ad esso